

جواب تمرین‌های سری دوم

پایند خوشانی

تمرین اول: آشنایی با Git و GitHub

سؤال اول: Git چیست و به چه دردی می‌خورد؟

ابزار Git به نرم‌افزار برای کنترل نسخه (Version Control) هست. یعنی هر تغییری که توی فایل‌های پروژه‌مون می‌دیم رو ثبت می‌کنه؛ به جورایی مثل دفترچه‌ی خاطرات کد. چندان از کاربردهاش:

- هر وقت خواستیم می‌تونیم برگردیم به یه نسخه‌ی قبلی از کارمون.
- معلومه کی، چه تغییری و کجای پروژه داده.
- چند نفر می‌تونن هم‌زمان روی یه پروژه کار کنن بدون این‌که کار همدیگه رو خراب کنن.
- هر بار یه مرحله از کار تموم می‌شه، یه Commit می‌زنیم؛ یعنی از وضعیت فعلی پروژه یه عکس می‌گیریم و ذخیره‌ش می‌کنیم.

سؤال دوم: GitHub چیست و چه فرقی با Git داره؟

سایت GitHub یه سرویس آنلاینه که پروژه‌هایی که با Git مدیریت می‌شن رو روی اینترنت نگه می‌داره. این جور یه کدمون فقط روی لپ‌تاپ خودمون نیست؛ از هر جایی بهش دسترسی داریم، می‌تونیم با بقیه به اشتراک بذاریم و با هم روش کار کنیم. فرقی‌شون به زبون ساده:

- ابزار Git یه نرم‌افزاره که روی کامپیوتر خودمون نصب می‌شه و بدون اینترنت هم کار می‌کنه.
 - سایت GitHub یه وب‌سایته که نسخه‌ی آنلاین پروژه‌مون اون‌جا ذخیره می‌شه.
 - ابزار Git رایگان و متن‌بازه و مال شرکت خاصی نیست؛ ولی سایت GitHub مال مایکروسافته.
 - سایت‌های دیگه‌ای مثل GitLab و Bitbucket هم همین کار GitHub رو می‌کنن.
- یه مثال: ابزار Git مثل دوربینیه که از هر مرحله‌ی کار عکس می‌گیره، و سایت GitHub مثل یه آلبوم آنلاین که عکس‌ها رو اون‌جا می‌ذاریم تا بقیه هم ببینن.

تمرین دوم: کلاس، صفت و رفتار

صفت (Property) یعنی ویژگی‌هایی که به چیز دارد، و رفتار (Method) یعنی کارهایی که اون چیز می‌تونه انجام بده. مثلاً به گریه رنگ داره (صفت) و میوه می‌کنه (رفتار).

پرانترز جلوی اسم رفتارها یعنی اون به متده.

مشتری (Customer)

مشتری اسم، شماره تماس و آدرس داره؛ می‌تونه ثبت‌نام کنه، سفارش بده، پول پرداخت کنه یا سفارشش رو لغو کنه.

Customer

Properties: Id, FirstName, LastName, PhoneNumber, Email, Address
Methods: Register(), PlaceOrder(), Pay(), CancelOrder()

دانشجو (Student)

دانشجو شماره‌ی دانشجویی، رشته و معدل داره؛ می‌تونه درس انتخاب کنه، سر کلاس بره، تکلیف تحویل بده و امتحان بده.

Student

Properties: StudentNumber, FirstName, LastName, Major, Average
Methods: EnrollCourse(), AttendClass(), SubmitHomework(), TakeExam()

استاد (Teacher)

استاد مدرک، رشته‌ی تخصصی و حقوق داره؛ درس می‌ده، حضور و غیاب می‌کنه، تکلیف می‌ده و برگه تصحیح می‌کنه.

Teacher

Properties: Id, FirstName, LastName, Degree, Field, Salary
Methods: Teach(), TakeAttendance(), GiveHomework(), GradeExam()

کارمند (Employee)

کارمند سمت، حقوق و تاریخ استخدام داره؛ ورودش رو ثبت می‌کنه، کار می‌کنه، مرخصی می‌گیره و حقوق دریافت می‌کنه.

Employee

Properties: EmployeeId, FirstName, LastName, Position, Salary, HireDate
Methods: CheckIn(), Work(), RequestLeave(), GetSalary()

مستطیل (Rectangle)

مستطیل طول، عرض و رنگ داره؛ می‌شه مساحت و محیطش رو حساب کرد یا اندازه‌ش رو تغییر داد.

Rectangle

Properties: Width, Height, Color
Methods: CalculateArea(), CalculatePerimeter(), Resize()

مربع (Square)

مربع فقط اندازه‌ی ضلع و رنگ داره، چون همه‌ی ضلع‌هاش برابر؛ رفتارهاش هم مثل مستطیله.

Square

Properties: Side, Color

Methods: CalculateArea(), CalculatePerimeter(), Resize()

سگ (Dog)

سگ اسم، نژاد، سن، رنگ و وزن دارد؛ پارس می‌کند، غذا می‌خورد، می‌دود و می‌خوابد.

Dog

Properties: Name, Breed, Age, Color, Weight

Methods: Bark(), Eat(), Run(), Sleep()

گربه (Cat)

گربه هم اسم، نژاد، سن، رنگ و وزن دارد؛ میو می‌کند، غذا می‌خورد، می‌خوابد و چنگ می‌زند.

Cat

Properties: Name, Breed, Age, Color, Weight

Methods: Meow(), Eat(), Sleep(), Scratch()

تمرین سوم: HTTP و HTTPS

سؤال اول: HTTP چیست؟

اسم کاملش HyperText Transfer Protocol هست. یه پروتکل؛ یعنی یه مجموعه قانون که مشخص می‌کند مرورگر و سرور چطوری با هم حرف بزنن، درخواست بفرستن و جواب بگیرن.

مشکلش اینه که اطلاعات رو بدون رمز و به صورت متن ساده جابه‌جا می‌کند. یعنی اگه کسی وسط راه (مثلاً روی Wi-Fi یه کافه) اطلاعات رو بگیره، راحت می‌تونه بخونه.

سؤال دوم: HTTPS چیست؟

همون HTTP هست به اضافه‌ی حرف S که یعنی Secure (امن). توی HTTPS اطلاعات قبل از فرستاده شدن رمزنگاری (Encrypt) می‌شن.

این رمزنگاری با چیزی به اسم TLS انجام می‌شه که قبلاً بهش SSL می‌گفتن. پس حتی اگه کسی وسط راه اطلاعات رو بگیره، فقط یه مشت حروف درهم و بی‌معنی می‌بینه.

سؤال سوم: فرقشون چیست؟

- از نظر امنیت: توی HTTP اطلاعات بدون رمز می‌رن، ولی توی HTTPS رمزنگاری می‌شن.
- از نظر آدرس: آدرس سایت با http یا https شروع می‌شه.
- از نظر پورت: معمولاً HTTP از پورت ۸۰ و HTTPS از پورت ۴۴۳ استفاده می‌کند.
- از نظر گواهی: برای HTTPS سایت باید یه گواهی امنیتی (SSL Certificate) داشته باشه.

- از نظر ظاهر: کنار آدرس سایت‌های HTTPS به علامت قفل می‌بینیم، ولی برای HTTP مرورگر عبارت Not secure رو نشون می‌ده.

سؤال چهارم: چرا امروزه بیشتر سایت‌ها از HTTPS استفاده می‌کنن؟

- چون از اطلاعات مهم مثل رمز عبور، شماره کارت و اطلاعات شخصی محافظت می‌کنه.
- چون مرورگرها سایت‌های HTTP رو با پیغام Not secure نشون می‌دن و کاربر بهشون اعتماد نمی‌کنه.
- چون Google به سایت‌های HTTPS توی نتایج جستجو امتیاز بیشتری می‌ده.
- چون خیلی از امکانات جدید مرورگر (مثل دسترسی به دوربین و موقعیت مکانی) فقط روی HTTPS کار می‌کنن.
- چون گرفتن گواهی امنیتی الان رایگان شده؛ مثلاً با سرویس Let's Encrypt.

تمرین چهارم: URL

سؤال اول: URL چیه؟

اسم کاملش Uniform Resource Locator هست. به زبون ساده، آدرس کامل یه چیز روی اینترنته؛ حالا اون چیز می‌تونه یه صفحه، یه عکس، یه ویدیو یا هر فایل دیگه‌ای باشه. همون چیزیه که بالای مرورگر توی نوار آدرس می‌بینیم.

سؤال دوم: بررسی قسمت‌های یه URL

این URL رو در نظر بگیریم:

`https://talabash.com/jewelry/women-gold-necklaces/`

توضیح هر قسمت:

- پروتکل (Scheme) که این‌جا `https` هست و مشخص می‌کنه با چه قانونی به سایت وصل بشیم. چون `https` هست، ارتباط با سایت امن و رمزنگاری شده‌ست.
- زیردامنه (Subdomain) توی این آدرس وجود نداره؛ یعنی مستقیماً وارد سایت اصلی می‌شیم. اگه آدرس با `www` شروع می‌شد، اون `www` زیردامنه بود.
- اسم دامنه (Domain Name) که این‌جا `talabash` هست و اسم اصلی سایته.
- پسوند دامنه (TLD) که این‌جا `com` هست. پسوندهای دیگه‌ای مثل `ir` و `org` و `de` هم داریم.
- پورت (Port) توی آدرس نوشته نشده، چون مرورگر خودش می‌دونه برای `https` باید از پورت پیش‌فرض ۴۴۳ استفاده کنه.

- مسیر (Path) که این جا jewelry/women-gold-necklaces هست و نشون می ده دقیقاً کدوم صفحه از سایت رو می خوایم. بخش اول یعنی دسته ی «جواهرات» و بخش دوم یعنی زیردسته ی «گردنبند طلای زنانه». علامت / آخر آدرس هم یعنی این مسیر یه دسته یا پوشه ست، نه یه فایل.
- پارامترها (Query String) توی این آدرس وجود ندارن، چون علامت سؤال توی آدرس نیست. اگه مثلاً محصولات رو فیلتر یا مرتب کنیم، ممکنه سایت چندتا پارامتر بعد از علامت سؤال به آدرس اضافه کنه.
- قطعه (Fragment) هم توی این آدرس نیست، چون علامت # نداره.

تمرین پنجم: GET و POST

سؤال اول: GET چیه؟

روش GET یعنی «یه چیزی بهم بده». وقتی می خوایم اطلاعاتی رو از سرور بگیریم و فقط ببینیم، از GET استفاده می کنیم. اطلاعات اضافه (مثل کلمه ای که جستجو کردیم) توی خود URL فرستاده می شه.

سؤال دوم: POST چیه؟

روش POST یعنی «اینو بگیر و ذخیره کن». وقتی می خوایم اطلاعاتی رو برای سرور بفرستیم (مثل ثبت نام، ورود به حساب یا ارسال فرم)، از POST استفاده می کنیم. اطلاعات توی بدنه ی درخواست (Body) فرستاده می شه و توی URL دیده نمی شه.

سؤال سوم: فرقشون چیه؟

- کار اصلی: روش GET برای گرفتن اطلاعات و روش POST برای فرستادن اطلاعات.
- جای اطلاعات: توی GET اطلاعات داخل URL هست و همه می بیننش؛ توی POST داخل بدنه ی درخواست.
- امنیت: برای رمز عبور هیچ وقت نباید از GET استفاده کرد، چون توی URL و تاریخچه ی مرورگر می مونه.
- حجم: طول URL محدودیت داره، پس GET برای اطلاعات زیاد مناسب نیست؛ ولی با POST می شه اطلاعات زیاد و حتی فایل فرستاد.
- ذخیره کردن: صفحه ای که با GET باز شده رو می شه توی Bookmark ذخیره کرد یا لینکش رو برای کسی فرستاد، ولی نتیجه ی POST رو نه.
- تکرار: اگه یه درخواست GET رو چند بار بفرستی اتفاقی نمی افته، ولی تکرار POST ممکنه مثلاً یه سفارش رو دو بار ثبت کنه.

سؤال چهارم: معمولاً کجا از GET استفاده می کنیم؟

- برای جستجو توی سایت ها.

- برای باز کردن یه صفحه یا دیدن یه محصول.
- برای دیدن لیست پست‌ها، فیلم‌ها یا محصولات.
- برای فیلتر کردن نتایج؛ مثلاً بر اساس قیمت یا رنگ.

سؤال پنجم: معمولاً کجا از POST استفاده می‌کنیم؟

- برای ثبت‌نام و ورود به حساب کاربری.
- برای ارسال فرم «تماس با ما».
- برای گذاشتن نظر یا پست جدید.
- برای فرستادن عکس یا فایل.
- برای ثبت سفارش و پرداخت.

مثال واقعی برای GET

جستجو توی Google: وقتی کلمه‌ی `csharp` رو جستجو می‌کنیم، این آدرس باز می‌شه و کلمه ما توی خود URL دیده می‌شه:

```
GET https://www.google.com/search?q=csharp
```

یه مثال از یه API واقعی: با این درخواست، اطلاعات عمومی یه کاربر `GitHub` رو می‌گیریم و جوابش به‌صورت JSON برمی‌گرده:

```
GET https://api.github.com/users/octocat
```

مثال واقعی برای POST

ورود به حساب توی Instagram: وقتی نام کاربری و رمزمون رو می‌نویسیم و دکمه‌ی ورود رو می‌زنیم، اطلاعات با POST فرستاده می‌شه. برای همینکه رمزمون توی نوار آدرس دیده نمی‌شه.

تمرین ششم: JSON و XML

سؤال اول: JSON چیه؟

اسم کاملش JavaScript Object Notation هست. یه روش ساده و سبک برای نوشتن و جابه‌جا کردن اطلاعات. اطلاعات به‌صورت جفت‌های «اسم و مقدار» (Key و Value) نوشته می‌شن و داخل آکولاد قرار می‌گیرن. با این‌که اسمش JavaScript داره، تقریباً همه‌ی زبان‌های برنامه‌نویسی، از جمله `C#`، باهاش کار می‌کنن.

سؤال دوم: XML چیست؟

اسم کاملش eXtensible Markup Language هست. اطلاعات رو با برچسب (Tag) می نویسه، خیلی شبیه HTML؛ با این فرق که اسم برچسبها رو خودمون انتخاب می کنیم. هر اطلاعاتی بین یه برچسب باز و یه برچسب بسته قرار می گیره.

سؤال سوم: چه کاربردی دارن؟

هر دو برای ذخیره کردن و جابه جا کردن اطلاعات بین برنامه ها هستن؛ مثلاً وقتی Client و Server با هم حرف می زنن یا وقتی یه API جواب می ده.

- فرمت JSON الان پرکاربردترین: توی بیشتر API ها، اپ های موبایل، وبسایت ها و فایل های تنظیمات (مثل appsettings.json توی پروژه های C#).
- فرمت XML بیشتر توی سیستم های قدیمی تر و بزرگ، سرویس های SOAP، فایل های RSS و فایل های تنظیمات استفاده می شه.

سؤال چهارم: چه فرقی های با هم دارن؟

- ظاهر: فرمت JSON با آکولاد و جفت «اسم و مقدار» نوشته می شه، ولی XML با برچسب های باز و بسته.
- حجم: فرمت JSON کوتاه تر و سبک تره، چون توی XML اسم هر برچسب دو بار تکرار می شه (یه بار باز، یه بار بسته).
- خوانایی: خوندن JSON برای آدم ها معمولاً راحت تره.
- نوع داده: فرمت JSON عدد، متن، true و false و لیست رو مستقیم می شناسه؛ ولی توی XML همه چی متنه.
- امکانات اضافه: فرمت XML می تونه ویژگی (Attribute) و توضیح (Comment) داشته باشه، ولی JSON توضیح نداره.
- سرعت: پردازش JSON معمولاً سریع تره و توی JavaScript خیلی راحت خونده می شه.

سؤال پنجم: اطلاعات یه Student با JSON و XML

نمایش با JSON :

```
{
  "studentId": 1001,
  "firstName": "Paniz",
  "lastName": "Khooshani",
  "major": "Multimedia",
  "university": "Pars University of Architecture and Art",
  "isActive": true,
  "courses": ["C# Programming", "Web Design"]
}
```

```
<?xml version="1.0" encoding="UTF-8"?>
<student>
  <studentId>1001</studentId>
  <firstName>Paniz</firstName>
  <lastName>Khooshani</lastName>
  <major>Multimedia</major>
  <university>Pars University of Architecture and Art</university>
  <isActive>true</isActive>
  <courses>
    <course>C# Programming</course>
    <course>Web Design</course>
  </courses>
</student>
```

توی XML برای لیست درس‌ها، هر درس رو داخل یه برچسب `course` جدا گذاشتیم. اگه دقت کنیم، XML برای همین اطلاعات حروف بیشتری لازم داشت؛ این همون فرق حجمیه که بالا گفتیم.